



BITS Pilani
Pilani Campus

Digital Design

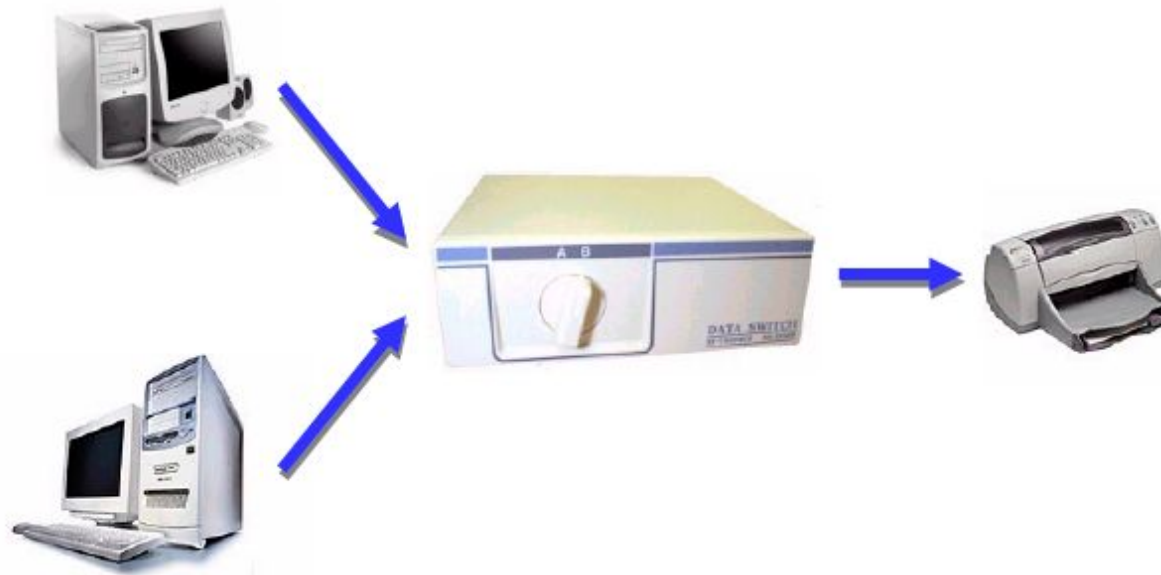
Prof. GSS Chalapathi,
Prof. Saurabh Gandhi,
Prof. Ramesh Sambangi

Review questions

1. A 3-to-8 decoder has how many inputs and outputs?
2. A decoder with 3 inputs (S_2 , S_1 , S_0) will have ____ AND gates with ____ inputs each; and ____ OR gates with ____ inputs each.
3. You want to build a circuit with 4 inputs and 6 outputs. The minimal number of components you will need is: ____ decoders (4-to-16); ____ AND gates; ____ OR gates
4. An active low decoder can be used to implement expressions in what form?

Multiplexers – digital traffic lights

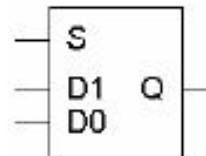
- In the old days, several machines could share an I/O device with a **switch**.



- The switch allows one computer's output to go to the printer's input.

2-to-1 multiplexer

- Here is the circuit analog of that printer switch.



- This is a 2-to-1 multiplexer, or mux.
 - There are two data inputs D0 and D1, and a select input called S.
 - There is one output named Q.
- The multiplexer routes one of its data inputs (D0 or D1) to the output Q, based on the value of S.
 - If $S=0$, the output will be D0.
 - If $S=1$, the output will be D1.

Building a multiplexer

- Here is a truth table for the multiplexer, based on our description from the previous page:

The multiplexer routes one of its data inputs (D0 or D1) to the output Q, based on the value of S.

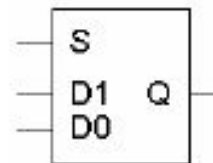
- If $S=0$, the output will be D0.
- If $S=1$, the output will be D1.

S	D1	D0	Q
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

- You can then find an MSP for the mux output Q.

$$Q = S'D0 + S D1$$

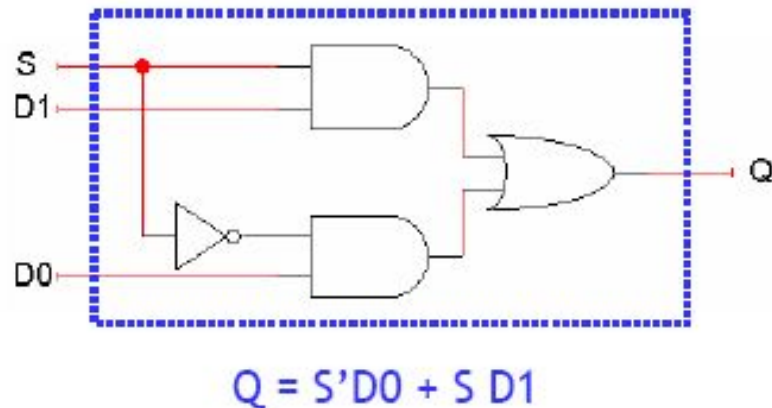
- Note that this corresponds closely to our English specification above—sometimes you can derive an expression without first making a truth table.



Building a multiplexer – the circuit



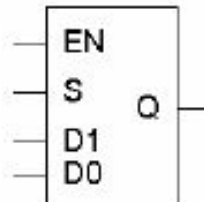
- Here is an implementation of a 2-to-1 multiplexer.



- Remember that a minimal sum of products expression leads to a minimal two-level circuit.

Multiplexers with *Enable* input

- Many devices have an additional **enable input**, which "activates" or "deactivates" the device.
- We could design a 2-to-1 multiplexer with an enable input that's used as follows.
 - EN=0 disables the multiplexer, which forces the output to be 0. (It does *not* turn off the multiplexer.)
 - EN=1 enables the multiplexer, and it works as specified earlier.
- Enable inputs are especially useful in combining smaller muxes together to make larger ones, as we'll see later today.



EN	S	D1	D0	Q
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	1
1	1	1	1	1

Truth table abbreviations

EN	S	D1	D0	Q
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	1
1	1	1	1	1

- Notice that when EN=0, then Q is always 0, regardless of what S, D1 and D0 are set to.
- We can shorten the truth table by including Xs in the input variable columns, as shown on the bottom right.



EN	S	D1	D0	Q
0	x	x	x	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	1
1	1	1	1	1

More abbreviations

- Also, when EN=1 notice that if $S=0$ then $Q=D0$, but if $S=1$ then $Q=D1$.
- Another way to abbreviate a truth table is to list input variables in the output columns, as shown on the right.

EN	S	D1	D0	Q
0	x	x	x	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	1
1	1	1	1	1



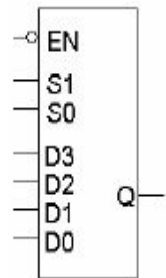
EN	S	Q
0	x	0
1	0	D0
1	1	D1

- This final version of the 2-to-1 multiplexer truth table is much clearer, and matches the equation $Q = S'D0 + S D1$ very closely.

4-to-1 mux



- Block diagram and truth table

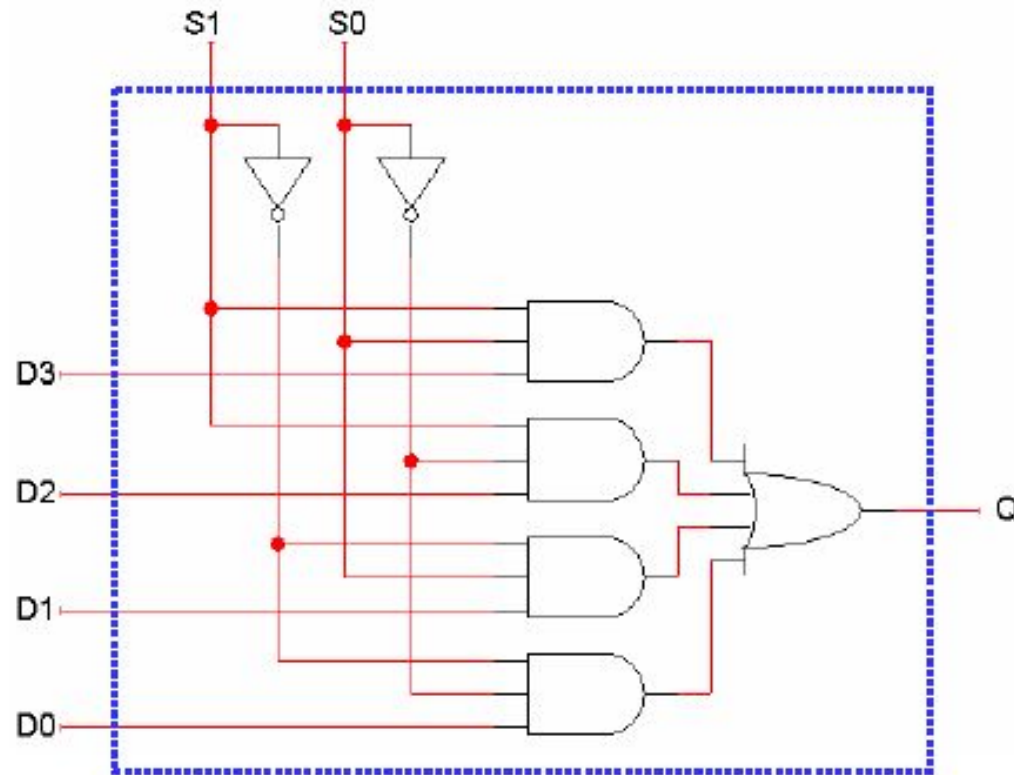


EN'	S1	S0	Q
0	0	0	D0
0	0	1	D1
0	1	0	D2
0	1	1	D3
1	x	x	1

- What is that bubble at the input of EN ?
- This is an active-low enable input multiplexer, i.e. when EN pin is made 0, output will select some input; when EN pin is 1, the multiplexer outputs 1 (***independent of the input***)
- Useful for hierarchical designs (we will see later)!

$$Q = S1'S0'D0 + S1'S0 D1 + S1 S0'D2 + S1 S0 D3$$

4-to-1 mux implementation

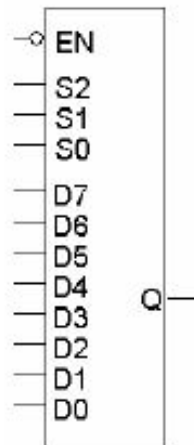


$$Q = S1'S0'D0 + S1'S0 D1 + S1 S0'D2 + S1 S0 D3$$

2^n -to-1 multiplexers



- You can make even larger multiplexers, following the same pattern.
- A 2^n -to-1 multiplexer routes one of 2^n input lines to the output line.
 - There are 2^n data inputs, so there must also be n select inputs.
 - The output is a single bit.
- Here is an 8-to-1 multiplexer, probably the biggest we'll see in this class.

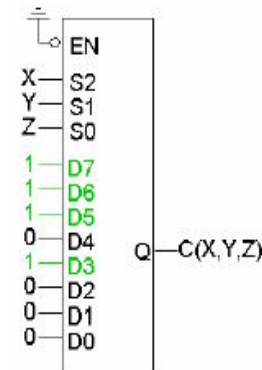


Implementing Boolean functions with muxes – addition



- n variable functions use an n -input multiplexer (n -to-1)
- The n variables connect to the *select* input pins of the mux, thereby selecting on row of the truth table
- *data* inputs are chosen intelligently to get the correct output

X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



- For each minterm to be included in the SOP, we connect the corresponding data input of the mux to 1; rest to 0

An adder with a smaller mux

- We can actually implement $C(X,Y,Z) = \Sigma m(3,5,6,7)$ with just a 4-to-1 mux, instead of an 8-to-1.

- First, we can divide the rows of our truth table into pairs, as shown on the right. X and Y are constant within each pair of rows, so C is a function of Z only.
 - When $XY=00$, $C=0$
 - When $XY=01$, $C=Z$
 - When $XY=10$, $C=Z$
 - When $XY=11$, $C=1$

X	Y	Z	C
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

An adder with a smaller mux



- All that's left is setting the multiplexer inputs.
 - The two input variables X and Y will be connected to select inputs $S1$ and $S0$ of our 4-to-1 multiplexer.
 - The expressions for $C(Z)$ are then connected to the data inputs $D0$ - $D3$ of the multiplexer.

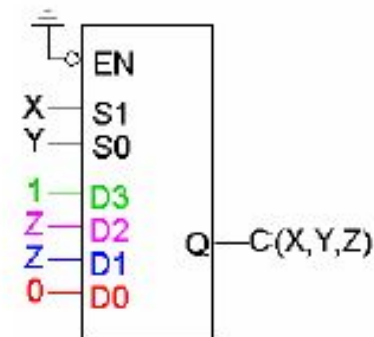
X	Y	Z	C
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

When $XY=00$, $C=0$

When $XY=01$, $C=Z$

When $XY=10$, $C=Z$

When $XY=11$, $C=1$



Adder with smaller mux – implement the sum



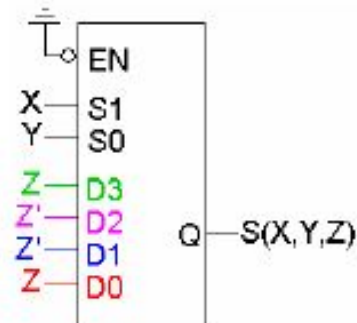
X	Y	Z	S
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

When $XY=00$, $S=Z$

When $XY=01$, $S=Z'$

When $XY=10$, $S=Z'$

When $XY=11$, $S=Z$

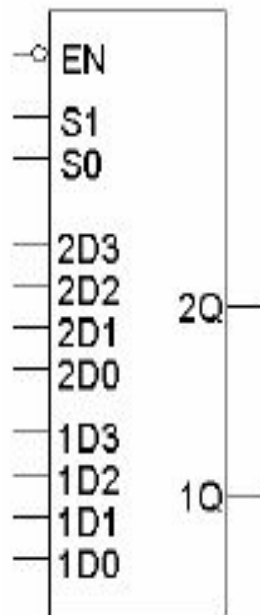


- Again, we can show that this is a correct implementation.

$$\begin{aligned} Q &= S1'S0'D0 + S1'S0 D1 + S1 S0'D2 + S1 S0 D3 \\ &= X'Y'Z + X'YZ' + XY'Z' + XYZ \\ &= \Sigma m(1,2,4,7) \end{aligned}$$

Dual multiplexers

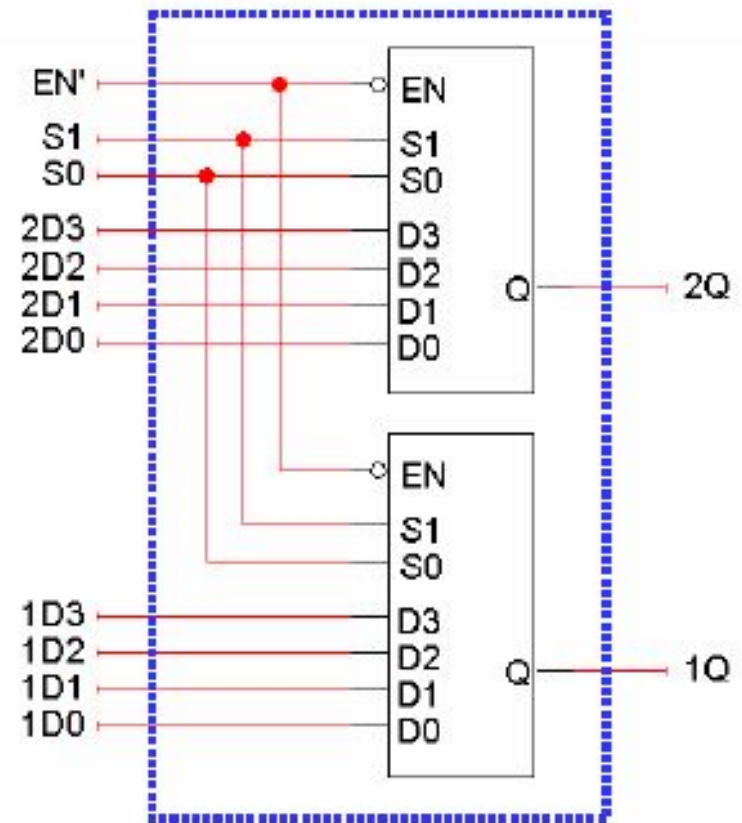
- Dual 4-to-1 mux: Instead of selecting 1 out of 4 input bits, selects 1 2-bit input out of 4 2-bit inputs



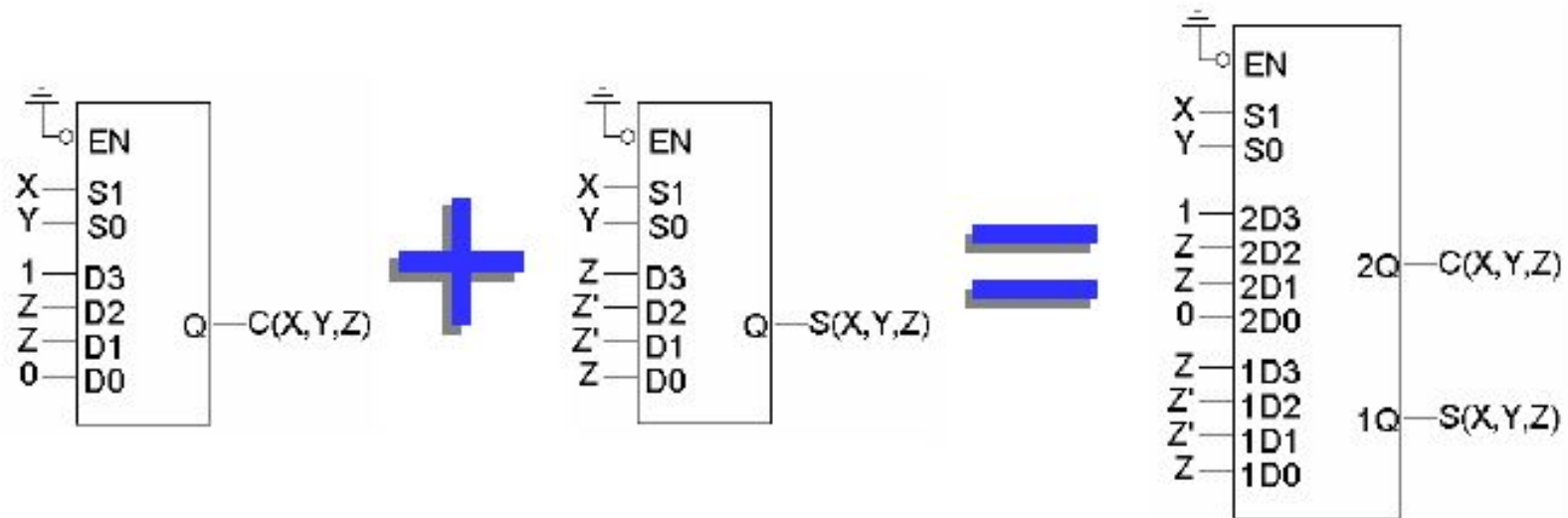
EN'	S1	S0	2Q	1Q
0	0	0	2D0	1D0
0	0	1	2D1	1D1
0	1	0	2D2	1D2
0	1	1	2D3	1D3
1	x	x	1	1

Building dual multiplexers

- You could build a dual 4-to-1 mux from its truth table and our familiar circuit design techniques.
- It's also possible to combine smaller muxes together to form larger ones.
- You can build the dual 4-to-1 mux just by using two 4-to-1 muxes.
 - The two 4-to-1 multiplexers share the same EN' , $S1$ and $S0$ signals.
 - Each smaller mux produces one bit of the two-bit output $2Q$ $1Q$.
- This kind of hierarchical design is very common in computer architecture.



Adder with dual mux – combining sum and carry in a single device



Example



Implement the Boolean function $F(A,B,C,D) = \Sigma (0,1,3,4,8,9,15)$

De-multiplexers

- Transmit information from a single line on to one of 2^n lines
- Selection of output line determined by n select lines
- A decoder with enable input can function as a demux

Finishing up combinational circuits!

Multipliers

- Multiplication can't be that hard! It's just repeated addition, so if we have adders, we should be able to do multiplication also.
- Here's an example of binary multiplication to get the mojo flowing.

$$\begin{array}{r}
 \begin{array}{rcccc}
 & & & 1 & 1 & 0 & 1 \\
 & & \times & 0 & 1 & 1 & 0 \\
 \hline
 & & & 0 & 0 & 0 & 0 \\
 & 1 & 1 & 0 & 1 & & \\
 & & 1 & 1 & 0 & 1 & \\
 + & 0 & 0 & 0 & 0 & & \\
 \hline
 & 1 & 0 & 0 & 1 & 1 & 1 & 0
 \end{array}
 \end{array}$$

Binary multiplication

				1	1	0	1	Multiplicand
				0	1	1	0	Multiplier
				0	0	0	0	} Partial products
			1	1	0	1		
		1	1	0	1			
		0	0	0	0			
+	0	0	0	0				
	1	0	0	1	1	1	0	Product

- Since we always multiply by either 0 or 1, the **partial products** are always either **0000** or the multiplicand (**1101** in this example).
- There are four partial products which are added to form the result.
 - We can add them in pairs, using three adders.
 - The product can have up to 8 bits, but we can use four-bit adders if we stagger them leftwards, like the partial products themselves.

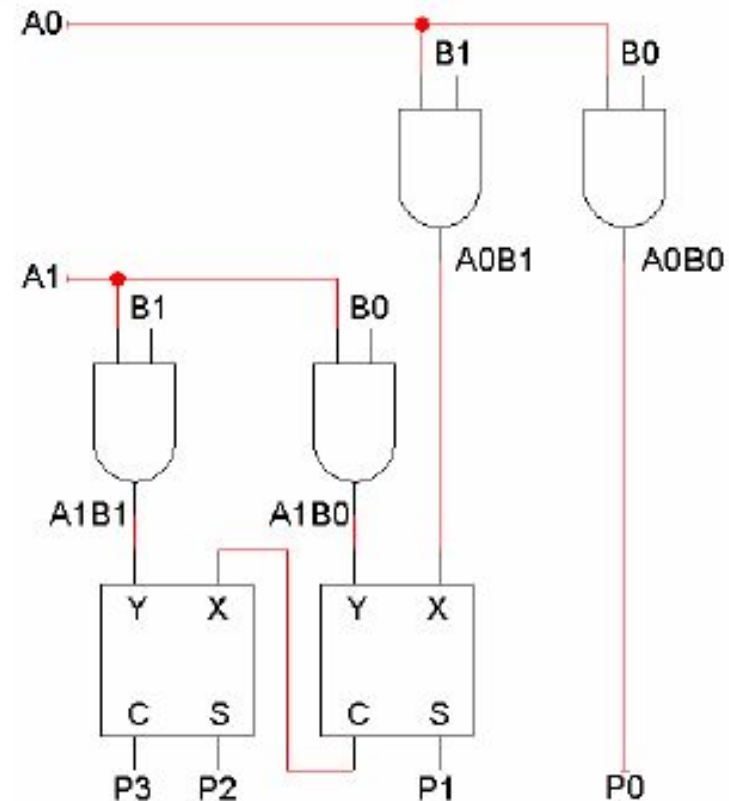
- $$\begin{array}{r}
 B1 B0 \\
 \times A1 A0 \\
 \hline
 A0B1 A0B0 \\
 + A1B1 A1B0 \\
 \hline
 P3 P2 P1
 \end{array}$$

- | A | B | $A \times B$ | $A \cdot B$ |
|---|---|--------------|-------------|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 |
| 1 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 |

2 x 2 multiplication



- Here is a circuit that multiplies the two-bit numbers $A1A0$ and $B1B0$, resulting in the four-bit product $P3P0$.
- For a 2×2 multiplier we can just use two half adders to sum the partial products. In general, though, we'll need full adders.
- The diagram on the next page shows how this can be extended to a four-bit multiplier, taking inputs $A3-A0$ and $B3-B0$ and outputting the product $P7-P0$.



4 x 4 multiplication

